

✓ Stage 2 — Data Acquisition & Filtering

Objective

Run all cells in notebook

The objective of this stage is to prepare the inspected dataset for subsequent analysis by identifying the required attributes and creating relevant filtered data views.

The dataset contains transaction, customer, banking, device, network, time and fraud-related attributes. Instead of modifying the original dataset, this stage creates controlled analysis views from the validated data.

Operations Performed

1. Created a working copy of the validated dataset.
2. Identified the attributes required for downstream analysis.
3. Examined the available transaction records for filtering.
4. Applied logical filters to identify valid transaction records.
5. Created separate analysis views for successful transactions and fraud-related records without modifying the original dataset.
6. Compared the original and filtered record counts.

Output

The original dataset is preserved, while filtered analysis-ready views are created for the next stages.

These outputs provide the required input for data extraction, validation, aggregation and analysis.

```
# =====  
# STAGE 2 — CONTAINER 1  
# WORKING DATASET ACQUISITION  
# =====  
  
# Create a separate working copy from the Stage 1 dataframe  
stage2_df = df.copy()  
  
print("Stage 2 working dataset created successfully.")  
print(f"Records : {stage2_df.shape[0]:,}")  
print(f"Columns : {stage2_df.shape[1]}")
```

```
Stage 2 working dataset created successfully.  
Records : 250,000  
Columns : 17
```

```
# =====  
# STAGE 2 — CONTAINER 2  
# REQUIRED ATTRIBUTE SELECTION  
# =====  
  
required_columns = [  
    "transaction id",  
    "timestamp",  
    "transaction type",  
    "merchant_category",  
    "amount (INR)",  
    "transaction_status",  
    "sender_age_group",
```

```

    "receiver_age_group",
    "sender_state",
    "sender_bank",
    "receiver_bank",
    "device_type",
    "network_type",
    "fraud_flag",
    "hour_of_day",
    "day_of_week",
    "is_weekend"
]

# Keep only columns that actually exist
available_columns = [
    column for column in required_columns
    if column in stage2_df.columns
]

transaction_view = stage2_df[available_columns].copy()

print("Required attribute selection completed.")
print(f"Selected columns : {len(available_columns)}")

display(
    pd.DataFrame({
        "Selected Attribute": available_columns
    })
)

```

Run all cells in notebook

Required attribute selection completed.
Selected columns : 17

	Selected Attribute 
0	transaction id
1	timestamp
2	transaction type
3	merchant_category
4	amount (INR)
5	transaction_status
6	sender_age_group
7	receiver_age_group
8	sender_state
9	sender_bank
10	receiver_bank
11	device_type
12	network_type
13	fraud_flag
14	hour_of_day
15	day_of_week
16	is_weekend

```
# =====
# STAGE 2 – CONTAINER 3
# POSITIVE TRANSACTION AMOUNT FILTER
# =====

amount_column = "amount (TND)"

# Convert the amount column safely to numeric
stage2_df[amount_column] = pd.to_numeric(
    stage2_df[amount_column],
    errors="coerce"
)

positive_amount_df = stage2_df[
    stage2_df[amount_column] > 0
].copy()

comparison = pd.DataFrame({
    "Dataset": [
        "Original Stage 2 Dataset",
        "Positive Amount Records",
        "Records Excluded"
    ],
    "Record Count": [
        len(stage2_df),
        len(positive_amount_df),
        len(stage2_df) - len(positive_amount_df)
    ]
})

display(comparison)
```

	Dataset	Record Count	
0	Original Stage 2 Dataset	250000	
1	Positive Amount Records	250000	
2	Records Excluded	0	

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```
# =====
# STAGE 2 – CONTAINER 4
# SUCCESSFUL TRANSACTION VIEW
# =====

status_column = "transaction_status"

# Inspect the actual status values present in the dataset
status_summary = (
    stage2_df[status_column]
    .value_counts(dropna=False)
    .rename_axis("Transaction Status")
    .reset_index(name="Record Count")
)

display(status_summary)

# Detect successful status values without assuming capitalization
status_text = (
    stage2_df[status_column]
    .astype(str)
```

```

        .str.strip()
        .str.lower()
    )

    successful_df = stage2_df[
        status_text.isin(["successful", "completed", "complete"])
    ].copy()

    print(f"Successful transaction records: {len(successful_df):,}")

```

	Transaction Status	Record Count	
0	SUCCESS	237624	
1	FAILED	12376	

Successful transaction records: 237,624

Next steps: [Generate code with status_summary](#) [New interactive sheet](#)

```

# =====
# STAGE 2 – CONTAINER 5
# FRAUD-RELATED RECORD VIEW
# =====

fraud_column = "fraud_flag"

# Display the actual values before filtering
fraud_summary = (
    stage2_df[fraud_column]
    .value_counts(dropna=False)
    .rename_axis("Fraud Flag")
    .reset_index(name="Record Count")
)

display(fraud_summary)

# Convert safely to numeric where possible
fraud_numeric = pd.to_numeric(
    stage2_df[fraud_column],
    errors="coerce"
)

fraud_df = stage2_df[
    fraud_numeric == 1
].copy()

print(f"Fraud-flagged records identified: {len(fraud_df):,}")

```

	Fraud Flag	Record Count	
0	0	249520	
1	1	480	

Fraud-flagged records identified: 480

Next steps: [Generate code with fraud_summary](#) [New interactive sheet](#)

```

# =====
# STAGE 2 – FINAL OUTPUT SUMMARY
# =====

stage2_summary = pd.DataFrame({

```

```

"Analysis View": [
    "Complete Working Dataset",
    "Positive Amount Transactions",
    "Successful Transactions",
    "Fraud-Flagged Transactions"
],
"Records": [
    len(stage2_df),
    len(positive_amount_df),
    len(successful_df),
    len(fraud_df)
],
"Columns": [
    stage2_df.shape[1],
    positive_amount_df.shape[1],
    successful_df.shape[1],
    fraud_df.shape[1]
]
})

display(stage2_summary)

```

Run all cells in notebook

	Analysis View	Records	Columns	
0	Complete Working Dataset	250000	17	
1	Positive Amount Transactions	250000	17	
2	Successful Transactions	237624	17	
3	Fraud-Flagged Transactions	480	17	

Next steps: [Generate code with stage2_summary](#) [New interactive sheet](#)

Stage 2 Conclusion

The validated dataset from Stage 1 was converted into a controlled working dataset without modifying the original records.

Relevant transaction attributes were identified and separate analysis views were created using logical filtering conditions.

Positive-amount transactions were isolated for transaction-value analysis, successful transactions were separated for completed-transaction analysis, and fraud-flagged records were isolated for subsequent fraud analysis.

The important outcome of this stage is not the deletion of records, but the creation of clearly defined data views for different analytical requirements.

These filtered datasets now provide structured inputs for Stage 3 — Data Extraction, where useful fields and derived information can be prepared for further processing.

Run all cells in notebook